AADK 9: Task 2 — Nullability, Safe Calls & Robust Data Handling

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – **Session 9**

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Research Key Concepts

Nullable Types (String?)

A **nullable type** is a variable that can hold either a value **or null**. In Kotlin, nullable types are declared using ? after the type. This helps prevent runtime errors by forcing developers to handle null cases explicitly.

Example:

val nickname: String? = null

Non-nullable Types (String)

A **non-nullable type** cannot hold null values. Kotlin ensures that these variables must always contain a valid value.

Example:

val username: String = "Rajat"

Safe Call Operator (?.)

The **safe call operator** allows a method or property to be called **only if the object is not null**. If the object is null, the expression returns null instead of crashing.

Example:

val name: String? = "Alex"

val length = name?.length

Elvis Operator (?:)

The **Elvis operator** provides a **default value when a nullable variable is null**.

Example:

val name: String? = null

val length = name?.length ?: 0

If name is null, the result becomes 0.

Non-null Assertion (!!)

The **non-null assertion operator** tells the compiler that a value **should not be null**. If it is null, the app will crash with a **NullPointerException**.

Example:

val name: String? = "John"

val length = name!!.length

This should be used carefully because it can cause crashes.

Example: Converting Nullable to Non-nullable Using if

```
val nickname: String? = null


val displayName: String =
    if (nickname != null) {
        nickname
    } else {
        "Guest"
    }
```

This ensures that the final value is always **non-null**.

---

2. Data Handling Scenario

Scenario: User Profile Fetch from Remote Server

In a mobile app, the user profile information is fetched from a **remote API**. The API returns a User? object where some fields such as photoUrl, nickname, or bio may be missing or null. Because network responses can be incomplete, the app must safely handle null values to prevent crashes and ensure the UI still displays meaningful information.

---

3. Null-safe Logic Document

Field Name	Type	Nullable?	Safe Handling Strategy	UI Fallback
nickname	String?	Yes	Use Elvis operator	Show "Guest"
photoUrl	String?	Yes	Check null before loading image	Placeholder image
bio	String?	Yes	Safe call operator	Show "No bio available"
email	String	No	Always available	Display normally

Example Data Class

```
data class User(
    val nickname: String?,
    val photoUrl: String?,
    val bio: String?,
    val email: String
)
```

Annotated Logic Plan

```
fun getDisplayName(user: User?): String {
    return user?.nickname ?: "Guest"
}
```

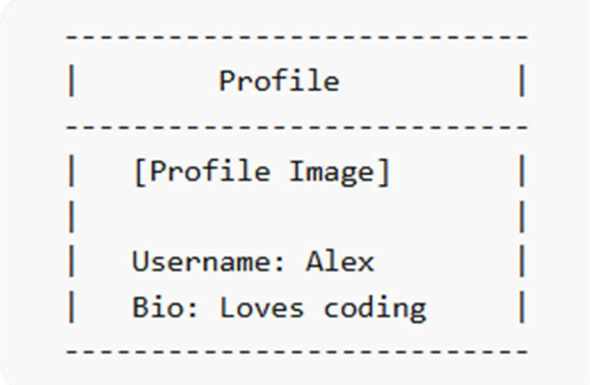
Explanation:

- user? ensures the user object is not null.
- nickname ?: "Guest" ensures a default value.

---

4. Compose UI Sketch & Modifier Plan

UI Sketch



Null Handling Example

- If photoUrl is null → show placeholder image
- If nickname is null → show **Guest**
- If bio is null → show **No bio available**

Compose Code Example

```
@Composable
fun ProfileScreen(user: User?) {

  Column {

    Text(
      text = user?.nickname ?: "Guest"
    )
```

```
        Text(
            text = user?.bio ?: "No bio available"
        )

    }
}
```

Image Handling Example

```
Image(
    painter = rememberAsyncImagePainter(
        user?.photoUrl ?: "placeholder_image_url"
    ),
    contentDescription = "Profile Image"
)
```

This ensures the app **always displays an image even when the URL is null**.

---

5. Reflection

Null safety is crucial for Android apps because many sources of data such as APIs, databases, and user inputs can return null values. If null values are not handled correctly, they can cause **NullPointerExceptions (NPEs)**, which may crash the app. Kotlin's null-safety system forces developers to explicitly handle null cases, making the application more stable. Thinking in null-safe types improves code reliability and reduces bugs. It also helps developers build apps that are safer, easier to maintain, and more user-friendly.